

Final Working Code

Introduction to Embedded System Design (IN 278)

Sourish Kundu (SR. No. - 26801)

Promit Chakraborty (SR. No. - 26000)

M.Tech – Instrumentation and Applied Physics
Indian Institute of Science (IISc)



Objective

The objective of this section is to demonstrate the complete implementation and real-time operation of a multi-peripheral embedded soil moisture monitoring system on the TI TMS320F28379D, integrating ADC-based sensing, filtered signal processing, PWM-driven actuation, and dual-display feedback using SPI (graphical TFT with pixelation-based visualization) and I2C (16×2 LCD for numerical output). The final working code validates synchronized operation of ADC, PWM, SPI, I2C, GPIO, and timer/interrupt mechanisms within a unified control loop, while also showcasing a novel visualization approach that enhances interpretability of sensor data beyond conventional display methods.

Final Working Code implemented in CCS

```
// ===== LIBRARIES =====
#include "F2837xD_device.h"
#include <stdio.h>
#include <stdlib.h> // Required for rand()

extern void DelayUs(Uint16);

// ===== DEFINES =====
// --- I2C LCD (External 1602) ---
#define LCD_ADDR 0x27
#define LCD_BACKLIGHT 0x08
#define EN 0x04
#define RS 0x01

// ===== GLOBALS =====
volatile Uint16 adcResult = 0;
float voltage = 0.0;
volatile float moisture = 0.0; // Marked volatile for ISR safety
float filtered_adc = 0.0;

char line1[16];
char line2[16];
int last_state = -1; // Tracks the size tier of the patch

// ===== FUNCTION PROTOTYPES =====
// System & Peripherals
void InitSysCtrl(void);
void configure_GPIO(void);
void configure_SPI(void);
void configure_I2C(void);
void configure_EPWM1(void);
void configure_CPU_timer0(void);
void configure_interrupts(void);
void initADC(void);
void initADCSOC(void);
```

```

// SPI LCD Functions (Updated for External ILI9341)
void LCD_reset_init(void);
void command_SPI_LCD(Uint16 command);
void data_SPI_LCD(Uint16 data);
void setAddrWindow(Uint16 x0, Uint16 y0, Uint16 x1, Uint16 y1);
void fillScreen(Uint16 color);
void drawPixelation(float moisture_level, int state);

// I2C LCD Functions
void I2C_LCD_Write(Uint16 data);
void LCD_Enable(Uint16 data);
void LCD_Write4Bits(Uint16 data);
void LCD_Command_I2C(Uint16 cmd);
void LCD_Data_I2C(Uint16 data);
void LCD_Init_I2C(void);
void LCD_Print(char *str);

// ISR
interrupt void timer_ISR(void);

// ===== MAIN =====
void main(void)
{
    EALLOW;
    WdRegs.WDCR.all = 0x0068; // Disable watchdog (CRITICAL:
        LEAVE DISABLED)
    EDIS;

    InitSysCtrl();

    // Initialize all hardware pins and modules
    configure_GPIO();
    configure_SPI();
    configure_I2C();
    configure_EPWM1();

    // Timer and Interrupt Setup
    configure_CPU_timer0();
    configure_interrupts();

    initADC();
    initADCSOC();

    DelayUs(20000);

    // Initialize both screens
    LCD_reset_init(); // External SPI
    LCD_Init_I2C();   // External I2C

    fillScreen(0x0000); // Clear SPI screen to black initially

```

```

// Start CPU timer for background PWM updates
CpuTimer0Regs.TCR.bit.TSS = 0;

while(1)
{
    // ----- 1. READ ADC -----
    AdcaRegs.ADCS0CFRC1.bit.SOCO = 1;
    while(AdcaRegs.ADCINTFLG.bit.ADCINT1 == 0);
    AdcaRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;

    adcResult = AdcaResultRegs.ADCRESULT0;

    // ----- 2. CALCULATE MOISTURE -----
    filtered_adc = (filtered_adc * 0.99f) + ((float)adcResult
        * 0.01f);
    voltage = (filtered_adc * 3.3f) / 4095.0f;

    float air_voltage = 1.14f;
    float water_voltage = 0.57f;

    moisture = ((air_voltage - voltage) / (air_voltage -
        water_voltage)) * 100.0f;

    if(moisture > 100.0f) moisture = 100.0f;
    if(moisture < 0.0f) moisture = 0.0f;

    // ----- 3. UPDATE I2C TEXT LCD -----
    sprintf(line1, "V=%5.2fV", voltage);
    sprintf(line2, "M=%6.1f%%", moisture);

    LCD_Command_I2C(0x80);
    LCD_Print(line1);
    LCD_Command_I2C(0xC0);
    LCD_Print(line2);

    // ----- 4. UPDATE SPI EXTERNAL LCD (SCALING
    //          PIXELATION) -----
    int current_state = 0;
    if (moisture >= 80.0f) {
        current_state = 3;        // Large
    } else if (moisture >= 60.0f) {
        current_state = 2;        // Medium
    } else if (moisture > 40.0f) {
        current_state = 1;        // Small
    } else {
        current_state = 0;        // Off
    }

    // Handle Cleanup
    if (current_state != last_state)
    {

```

```

        if (current_state < last_state)
        {
            setAddrWindow(80, 40, 239, 199);
            Uint32 i;
            for(i=0; i<25600; i++) {
                data_SPI_LCD(0x00);
                data_SPI_LCD(0x00);
            }
        }
        last_state = current_state;
    }

    // Draw the pixelation if active
    if (current_state > 0)
    {
        drawPixelation(moisture, current_state);
    }

    // ----- 5. DELAY -----
    int d;
    for(d=0; d<4; d++) {
        DelayUs(50000);
    }
}

// ===== ISR (BACKGROUND PWM UPDATE) =====
interrupt void timer_ISR(void)
{
    // The PWM duty logic has been moved here. It will run
    // independently
    // based on the Timer0 period, using the latest global
    // moisture value.
    float duty = moisture / 100.0f;

    if(duty < 0.05f) duty = 0.05f; // minimum visibility

    Uint16 cmp = (Uint16)(duty * EPwm1Regs.TBPRD);

    EPwm1Regs.CMPB.bit.CMPB = cmp; // LED brightness
    EPwm1Regs.CMPA.bit.CMPA = cmp; // Buzzer intensity

    // Acknowledge the interrupt
    PieCtrlRegs.PIEACK.all = 0x0001;
}

// ===== HARDWARE CONFIGURATIONS =====
void InitSysCtrl(void)
{
    EALLOW;

```

```

    ClkCfgRegs.CLKSRCCTL1.bit.OSCCLKSRCSEL = 1;
    ClkCfgRegs.SYSPLLCTL1.bit.PLLCLKEN = 0;
    ClkCfgRegs.SYSCLKDIVSEL.all = 2;
    ClkCfgRegs.SYSPLLMULT.all = 20;

    while(ClkCfgRegs.SYSPLLSTS.bit.LOCKS != 1);

    ClkCfgRegs.SYSCLKDIVSEL.all = 1;
    ClkCfgRegs.SYSPLLCTL1.bit.PLLCLKEN = 1;
    ClkCfgRegs.SYSCLKDIVSEL.all = 0;
    EDIS;
}

void configure_GPIO(void)
{
    EALLOW;

    // --- I2C PINS ---
    GpioCtrlRegs.GPDGMUX1.bit.GPIO105 = 0;
    GpioCtrlRegs.GPDMUX1.bit.GPIO105 = 1; // I2C SCL
    GpioCtrlRegs.GPDPU1.bit.GPIO105 = 0;

    GpioCtrlRegs.GPDGMUX1.bit.GPIO104 = 0;
    GpioCtrlRegs.GPDMUX1.bit.GPIO104 = 1; // I2C SDA
    GpioCtrlRegs.GPDPU1.bit.GPIO104 = 0;

    // --- PWM PINS ---
    GpioCtrlRegs.GPAGMUX1.bit.GPIO0 = 0;
    GpioCtrlRegs.GPAMUX1.bit.GPIO0 = 1; // EPWM1A
    GpioCtrlRegs.GPAGMUX1.bit.GPIO1 = 0;
    GpioCtrlRegs.GPAMUX1.bit.GPIO1 = 1; // EPWM1B

    // --- SPI LCD PINS ---
    GpioCtrlRegs.GPBGMUX2.bit.GPIO58 = 3;
    GpioCtrlRegs.GPBMUX2.bit.GPIO58 = 3; // SPI SIMO

    GpioCtrlRegs.GPBGMUX2.bit.GPIO60 = 3;
    GpioCtrlRegs.GPBMUX2.bit.GPIO60 = 3; // SPI CLK

    // SPI CS -> GPIO124
    GpioCtrlRegs.GPDGMUX2.bit.GPIO124 = 0;
    GpioCtrlRegs.GPDMUX2.bit.GPIO124 = 0; // GPIO mode
    GpioCtrlRegs.GPDDIR.bit.GPIO124 = 1; // Output
    GpioDataRegs.GPDSET.bit.GPIO124 = 1; // Idle High

    // SPI RST -> GPIO122
    GpioCtrlRegs.GPDGMUX2.bit.GPIO122 = 0;
    GpioCtrlRegs.GPDMUX2.bit.GPIO122 = 0; // GPIO mode
    GpioCtrlRegs.GPDDIR.bit.GPIO122 = 1; // Output
    GpioDataRegs.GPDSET.bit.GPIO122 = 1; // Idle High

```

```

    // SPI DC/RS -> GPIO24
    GpioCtrlRegs.GPAGMUX2.bit.GPIO24 = 0;
    GpioCtrlRegs.GPAMUX2.bit.GPIO24 = 0;    // GPIO mode
    GpioCtrlRegs.GPADIR.bit.GPIO24 = 1;    // Output

    EDIS;
}

void configure_SPI(void)
{
    EALLOW;
    CpuSysRegs.PCLKCR8.bit.SPI_A = 1;
    asm("_NOP"); asm("_NOP");

    SpiaRegs.SPICCR.bit.SPISWRESET = 0;
    SpiaRegs.SPICTL.bit.MASTER_SLAVE = 1;
    SpiaRegs.SPICCR.bit.SPICHAR = 7;
    SpiaRegs.SPICCR.bit.CLKPOLARITY = 1; // 1 preferred for
        ILI9341
    SpiaRegs.SPICTL.bit.CLK_PHASE = 0;
    SpiaRegs.SPIBRR.bit.SPI_BIT_RATE = 49;
    SpiaRegs.SPIPRI.bit.TRIWIRE = 0;
    SpiaRegs.SPICCR.bit.SPISWRESET = 1;
    EDIS;
}

void configure_CPU_timer0(void)
{
    EALLOW;
    CpuTimer0Regs.PRD.all = 199999999;
    CpuTimer0Regs.TCR.bit.TSS = 1; // Stop timer during config
    CpuTimer0Regs.TCR.bit.TRB = 1;
    CpuTimer0Regs.TCR.bit.TIE = 1;
    EDIS;
}

void configure_interrupts(void)
{
    DINT;
    PieCtrlRegs.PIECTRL.bit.ENPIE = 1;

    // CRITICAL: Must be mapped with EALLOW since PieVectTable is
        protected
    EALLOW;
    PieVectTable.TIMER0_INT = &timer_ISR;
    EDIS;

    PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
    IER = M_INT1;
    EINT;
}

```

```

void configure_I2C(void)
{
    EALLOW;
    DevCfgRegs.CPUSEL7.bit.I2C_A = 0;
    CpuSysRegs.PCLKCR9.bit.I2C_A = 1;

    I2caRegs.I2CMDR.bit.IRS = 0;
    I2caRegs.I2CPSC.bit.IPSC = 19;
    I2caRegs.I2CCLKL = 45;
    I2caRegs.I2CCLKH = 45;
    I2caRegs.I2CMDR.bit.IRS = 1;
    EDIS;
}

void configure_EPWM1(void)
{
    EALLOW;
    CpuSysRegs.PCLKCR2.bit.EPWM1 = 1;
    CpuSysRegs.PCLKCR0.bit.TBCLKSYNC = 0;

    EPwm1Regs.TBPRD = 2000;
    EPwm1Regs.TBCTL.bit.CTRMODE = 0;

    EPwm1Regs.AQCTLA.bit.ZRO = 2; // Buzzer
    EPwm1Regs.AQCTLA.bit.CAU = 1;

    EPwm1Regs.AQCTLB.bit.ZRO = 2; // LED
    EPwm1Regs.AQCTLB.bit.CBU = 1;

    CpuSysRegs.PCLKCR0.bit.TBCLKSYNC = 1;
    EDIS;
}

void initADC(void)
{
    EALLOW;
    CpuSysRegs.PCLKCR13.bit.ADC_A = 1;
    AdcaRegs.ADCCTL2.bit.PRESCALE = 6;
    AdcaRegs.ADCCTL1.bit.ADCPDNZ = 1;
    DelayUs(1000);
    EDIS;
}

void initADCSOC(void)
{
    EALLOW;
    AdcaRegs.ADCSOCCTL.bit.CHSEL = 0; // ADCINA0
    AdcaRegs.ADCSOCCTL.bit.ACQPS = 14;
    AdcaRegs.ADCINTSEL1N2.bit.INT1SEL = 0;
    AdcaRegs.ADCINTSEL1N2.bit.INT1E = 1;
}

```

```

    EDIS;
}

// ===== SPI EXTERNAL LCD FUNCTIONS =====
void command_SPI_LCD(Uint16 command)
{
    GpioDataRegs.GPDCLEAR.bit.GPIO124 = 1; // CS LOW
    GpioDataRegs.GPACLEAR.bit.GPIO24 = 1; // DC LOW

    SpiaRegs.SPICTL.bit.TALK = 1;
    SpiaRegs.SPITXBUF = (Uint16)((Uint32)command << 8);

    while(SpiaRegs.SPISTS.bit.INT_FLAG == 0);
    (void)(SpiaRegs.SPIRXBUF);

    GpioDataRegs.GPDSET.bit.GPIO124 = 1; // CS HIGH
}

void data_SPI_LCD(Uint16 data)
{
    GpioDataRegs.GPDCLEAR.bit.GPIO124 = 1; // CS LOW
    GpioDataRegs.GPASET.bit.GPIO24 = 1; // DC HIGH

    SpiaRegs.SPICTL.bit.TALK = 1;
    SpiaRegs.SPITXBUF = (Uint16)((Uint32)data << 8);

    while(SpiaRegs.SPISTS.bit.INT_FLAG == 0);
    (void)(SpiaRegs.SPIRXBUF);

    GpioDataRegs.GPDSET.bit.GPIO124 = 1; // CS HIGH
}

void setAddrWindow(Uint16 x0, Uint16 y0, Uint16 x1, Uint16 y1)
{
    command_SPI_LCD(0x2A);
    data_SPI_LCD(x0 >> 8); data_SPI_LCD(x0 & 0xFF);
    data_SPI_LCD(x1 >> 8); data_SPI_LCD(x1 & 0xFF);

    command_SPI_LCD(0x2B);
    data_SPI_LCD(y0 >> 8); data_SPI_LCD(y0 & 0xFF);
    data_SPI_LCD(y1 >> 8); data_SPI_LCD(y1 & 0xFF);

    command_SPI_LCD(0x2C);
}

void fillScreen(Uint16 color)
{
    Uint32 i;
    setAddrWindow(0, 0, 319, 239);
    for(i=0; i < (Uint32)320 * 240; i++)
    {

```

```

        data_SPI_LCD(color >> 8);
        data_SPI_LCD(color & 0xFF);
    }
}

void LCD_reset_init(void)
{
    GpioDataRegs.GPDCLEAR.bit.GPIO122 = 1;
    DelayUs(20000);
    GpioDataRegs.GPDSET.bit.GPIO122 = 1;

    int i;
    for(i=0; i<4; i++) { DelayUs(50000); }

    command_SPI_LCD(0x11); // Sleep Out
    for(i=0; i<4; i++) { DelayUs(50000); }

    command_SPI_LCD(0x36); // MAC
    data_SPI_LCD(0x28);    // Landscape

    command_SPI_LCD(0x3A); // Color Format
    data_SPI_LCD(0x55);    // 16-bit

    fillScreen(0x0000);

    command_SPI_LCD(0x29); // Display On
}

// DYNAMIC PIXELATION GENERATOR
void drawPixelation(float moisture_level, int state)
{
    Uint32 i, max_pixels;
    int threshold = (int)moisture_level;

    if (state == 1) {
        // Small: 80x80 patch
        setAddrWindow(120, 80, 199, 159);
        max_pixels = 6400;
    }
    else if (state == 2) {
        // Medium: 120x120 patch
        setAddrWindow(100, 60, 219, 179);
        max_pixels = 14400;
    }
    else {
        // Large: 160x160 patch
        setAddrWindow(80, 40, 239, 199);
        max_pixels = 25600;
    }

    for(i = 0; i < max_pixels; i++)

```

```

    {
        if ((rand() % 100) < threshold)
        {
            Uint16 random_color = rand();
            data_SPI_LCD(random_color >> 8);
            data_SPI_LCD(random_color & 0xFF);
        }
        else
        {
            data_SPI_LCD(0x00);
            data_SPI_LCD(0x00);
        }
    }
}

// ===== I2C EXTERNAL LCD FUNCTIONS =====
void I2C_LCD_Write(Uint16 data)
{
    while(I2caRegs.I2CMDR.bit.STP == 1);
    I2caRegs.I2CSAR.bit.SAR = LCD_ADDR;
    I2caRegs.I2CCNT = 1;
    I2caRegs.I2CDXR.bit.DATA = data;
    I2caRegs.I2CMDR.all = 0x6E20;
    while(I2caRegs.I2CMDR.bit.STP == 1);
}

void LCD_Enable(Uint16 data)
{
    I2C_LCD_Write(data | EN);
    DelayUs(1000);
    I2C_LCD_Write(data & ~EN);
}

void LCD_Write4Bits(Uint16 data)
{
    I2C_LCD_Write(data | LCD_BACKLIGHT);
    LCD_Enable(data | LCD_BACKLIGHT);
}

void LCD_Command_I2C(Uint16 cmd)
{
    Uint16 high = (cmd & 0xF0);
    Uint16 low  = ((cmd << 4) & 0xF0);
    LCD_Write4Bits(high);
    LCD_Write4Bits(low);
}

void LCD_Data_I2C(Uint16 data)
{
    Uint16 high = (data & 0xF0) | RS;
    Uint16 low  = ((data << 4) & 0xF0) | RS;
}

```

```

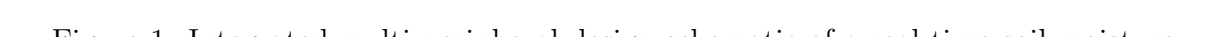
        LCD_Write4Bits(high);
        LCD_Write4Bits(low);
    }

void LCD_Init_I2C(void)
{
    DelayUs(20000);
    LCD_Write4Bits(0x30);
    DelayUs(5000);
    LCD_Write4Bits(0x30);
    DelayUs(1000);
    LCD_Write4Bits(0x30);
    DelayUs(1000);
    LCD_Write4Bits(0x20);

    LCD_Command_I2C(0x28);
    LCD_Command_I2C(0x0C);
    LCD_Command_I2C(0x06);
    LCD_Command_I2C(0x01);
    DelayUs(2000);
}

void LCD_Print(char *str)
{
    while(*str)
    {
        LCD_Data_I2C(*str++);
    }
}

```



Code executed in CCS

- Executed Fully Working Code in CCS

Video Demonstration Link

- Video Demonstration Link

References from Technical Reference Manuals

Table 5. F28379D LaunchPad Pin Out and Pin Mux Options - J1, J3

Mux Value				J1 Pin	J3 Pin	Mux Value			
X	2	1	0			0	Alt Function	2	X
			3.3V	1	21	5V			
			GPIO32	2	22	GND			
	SCIRXDB		GPIO19	3	23	ADCIN14	CMPIN4P		
	SCITXDB		GPIO18	4	24	ADCINC3	CMPIN6N		
			GPIO67	5	25	ADCINB3	CMPIN3N		
			GPIO111	6	26	ADCINA3	CMPIN1N		
SPICLKA ⁽¹⁾			GPIO60	7	27	ADCINC2	CMPIN6P		
			GPIO22	8	28	ADCINB2	CMPIN3P		
		SCLA	GPIO105 ⁽²⁾	9	29	ADCINA2	CMPIN1P		
		SDAA	GPIO104 ⁽²⁾	10	30	ADCINA0	DACOUTA		

Figure 2: F28379D LaunchPad Pin Out and Pin Mux Options - J1, J3 -
"sprui77c-LaunchPadUserGuide"

Table 6. F28379D LaunchPad Pin Out and Pin Mux Options - J4, J2

Mux Value				J4 Pin	J2 Pin	Mux Value			
X	2	1	0			0	1	2	X
		EPWM1A	GPIO0	40	20	GND			
		EPWM1B	GPIO1	39	19	GPIO61			
		EPWM2A	GPIO2	38	18	GPIO123			SD1_C1 ⁽¹⁾
		EPWM2B	GPIO3	37	17	GPIO122			SD1_D1 ⁽¹⁾
		EPWM3A	GPIO4	36	16	RST			
		EPWM3B	GPIO5	35	15	GPIO58			SPISIMOA ⁽¹⁾
		OUTPUTXBAR1	GPIO24	34	14	GPIO59			SPISOMIA ⁽¹⁾
OUTPUTXBAR7 ⁽¹⁾			GPIO16	33	13	GPIO124			SD1_D2 ⁽¹⁾
			DAC1	32	12	GPIO125			SD1_C2 ⁽¹⁾
			DAC2	31	11	GPIO29 ⁽²⁾			OUTPUTXBAR6 ⁽¹⁾

Figure 3: F28379D LaunchPad Pin Out and Pin Mux Options - J4, J2 -
"sprui77c-LaunchPadUserGuide"

Pinout Diagram for your BoosterPack

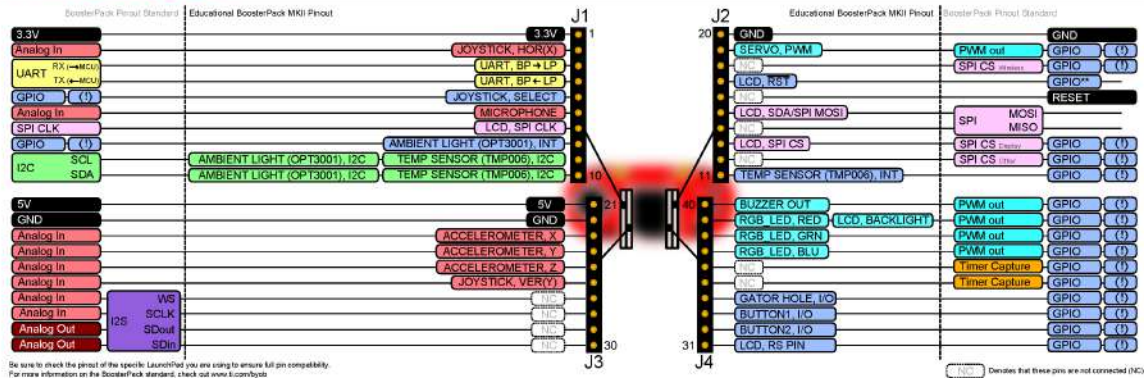


Figure 4: BoosterPack Plug-in Module Pinout - "slau599b"

Design Details

Subsystem	Functionality	GPIO	BoosterPack Pin
2.4inch SPI Module ILI9341	SPI SIMO	58	J2-15
	SPI CLK	60	J1-07
	LCD CS	124	J2-13
	LCD RST	122	J2-17
	LCD RS/DC	24 (J4-34 → J4-31)	J4-31
	LED (+ 3.3 V)	-	J1-01
	VCC (+ 5 V)	-	J3-21
	GND	-	J3-22
PWM configured External LED	GND	-	J3-22
	External LED	GPIO1 (EPWM1B)	J4.39
Analog Interface + H 1635 Soil Humidity Sensor	GND	GND	J3.22
	+5V	+5V	J3.21
	ADCINA0	ADCINA0	J3.30
I2C Interface (PCF8574) + External LCD (16x2)	GND	GND	J3.22
	+5V	+5V	J3.21
	SDA	GPIO104	J1.10
	SCL	GPIO105	J1.9

Table 1: Pin Mapping for External SPI configured LCD screen, External I2C configured LCD screen, PWM, ADC, and I2C Interfaces on TI F28379D for the Real Time Soil Moisture Monitoring System

Peripheral Utilization Summary

- **ADC:** Acquires real-time analog soil moisture data from the H1635 sensor via ADCINA0.
- **System Clock:** Provides deterministic timing and synchronization for all on-chip operations.
- **GPIO:** Enables digital interfacing and control signal routing across peripherals.
- **SPI:** Interfaces the 2.4-inch ILI9341 display module for high-speed graphical output in accordance with moisture level.
- **I2C:** Communicates with the external 16×2 LCD (via PCF8574) for textual display.
- **PWM:** Generates duty-cycle controlled signals to drive the external LED for additional indication.
- **Timer:** Establishes periodic sampling and control intervals for real-time operation.
- **Interrupt (ISR):** Implements a timer-driven control loop to update PWM and to drive the external LED for additional indication.

System Overview and Functional Description

- **ADC:** Samples the analog voltage from the H1635 soil moisture sensor and converts it into a 12-bit digital value for real-time processing.
- **I2C:** Implements a two-wire serial interface to transmit processed voltage and computed moisture percentage to the external 16×2 LCD (via PCF8574).
- **SPI:** Drives the external ILI9341 TFT display by rendering a dynamic pixel patch, where patch size varies with moisture level:
 - Small Patch (40–60%): 80×80 pixels
 - Medium Patch (60–80%): 120×120 pixels
 - Large Patch (80–100%): 160×160 pixels
- **PWM:** Utilizes EPWM1B (mapped to GPIO1) to generate a duty-cycle controlled signal for LED brightness modulation, with duty updates performed via the CMPB register enabling smooth hardware-driven intensity variation.
- **ISR (Interrupt Service Routine):** A timer-triggered ISR executes periodically (1 Hz), converting moisture percentage into corresponding PWM compare values and updating CMPA/CMPB registers, enabling autonomous real-time control through hardware PWM.
- **Calibration Challenge:** The H1635 sensor exhibits a very narrow operating voltage range (≈ 0.03 V), with observed saturation between 2.68 V (wet) and 2.71 V (dry), resulting in poor sensitivity and premature saturation of computed moisture values near 100%.

- **Implication:** Due to the limited voltage variation, the ADC resolution is under-utilized, leading to reduced measurement accuracy and necessitating calibration enhancement or signal conditioning.

Sensor Calibration and Moisture Estimation

Purpose: To convert the analog voltage output of the H1635 soil humidity sensor into a meaningful moisture percentage using calibrated reference points and signal processing.

Voltage Acquisition: The sensor output is first digitized using the 12-bit ADC (range 0–4095). The corresponding analog voltage is computed as:

$$V = \frac{\text{ADC}_{\text{value}} \times 3.3}{4095}$$

To reduce measurement noise and improve stability, an exponential smoothing filter is applied:

$$V_f = 0.99 \cdot V_{\text{prev}} + 0.01 \cdot V_{\text{new}}$$

Calibration Method: The sensor is calibrated using two experimentally determined reference voltages:

- $V_{\text{air}} = 1.14 \text{ V}$ (dry soil condition)
- $V_{\text{water}} = 0.57 \text{ V}$ (fully wet condition)

Using these reference points, the moisture percentage is computed using linear interpolation:

$$\text{Moisture} = \frac{V_{\text{air}} - V}{V_{\text{air}} - V_{\text{water}}} \times 100$$

Saturation Handling: To ensure physically meaningful values, the computed moisture is bounded within the range:

$$0 \leq \text{Moisture} \leq 100$$

Key Insight: The calibration relies on the inverse relationship between sensor voltage and moisture level, where voltage decreases as moisture content increases. Filtering and normalization are essential to obtain stable measurements from the ADC output.

Design Note: Although the updated calibration provides improved mapping across the operating range, the overall voltage variation remains limited. This restricts effective utilization of ADC resolution and indicates the potential need for signal conditioning (e.g., amplification) in future improvements.

Experimental Components and Setup



Figure 5: Top and bottom views of the H1635 soil moisture sensor module showing electrode structure and onboard signal conditioning circuitry.

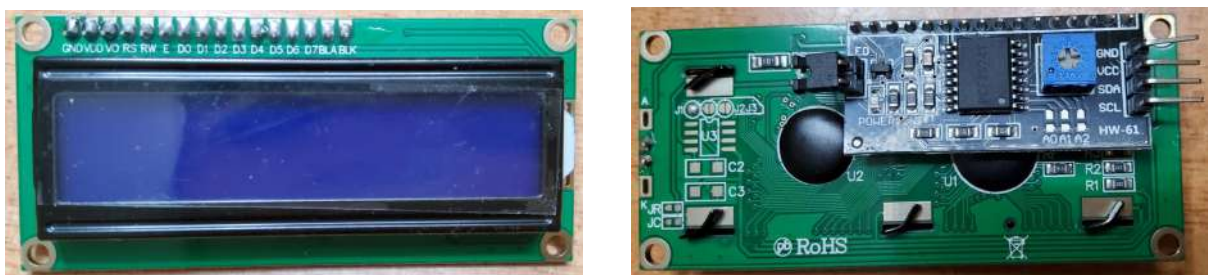


Figure 6: 16×2 LCD module and its I2C interface (PCF8574 backpack), illustrating the conversion from parallel LCD pins to a two-wire serial communication interface.

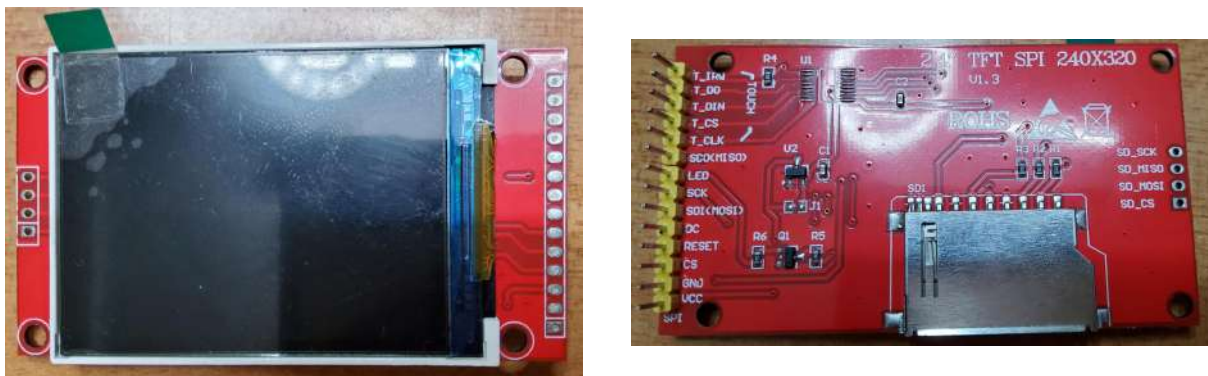


Figure 7: Front and back views of the 2.4-inch SPI TFT display module (240×320), showing the display panel and underlying SPI interface circuitry with pin configuration.

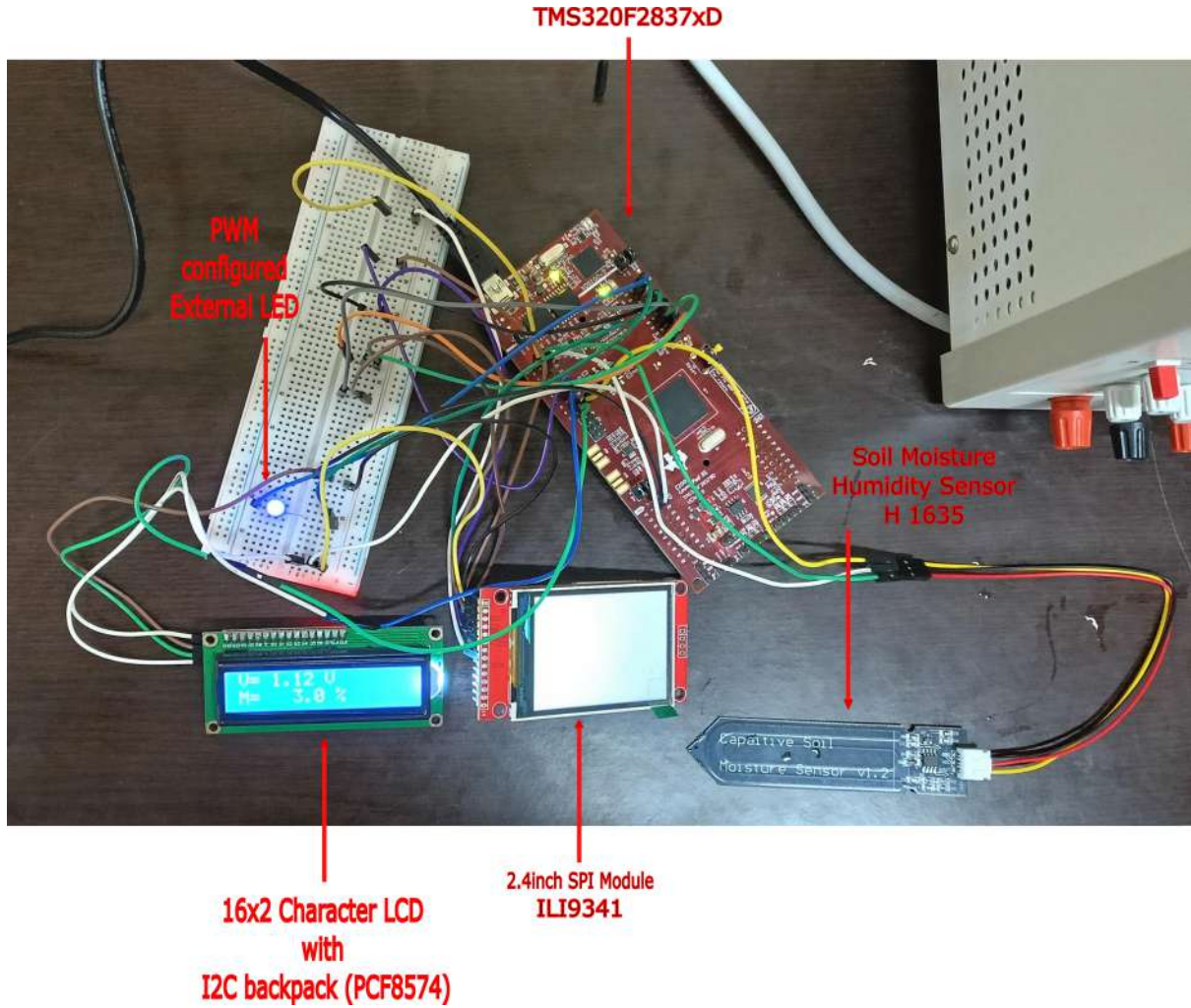


Figure 8: Experimental Setup

Conclusion

The implemented system successfully demonstrates a real-time embedded soil moisture monitoring and feedback solution on the TI TMS320F28379D by integrating multiple peripherals including ADC, SPI, I2C, PWM, GPIO, and timer-driven interrupts. The design achieves synchronized sensing, processing, actuation, and dual-mode visualization through both numerical display and a novel pixelation-based graphical interface. The use of filtering and calibration ensures stable measurement, while hardware-driven PWM enables efficient and continuous control. Although the system is functionally robust, the limited voltage variation of the H1635 sensor constrains measurement sensitivity, indicating scope for improved signal conditioning and calibration in future enhancements. Overall, the project establishes a scalable and extensible framework for intelligent monitoring and control applications.

Contribution

- **Promit Chakraborty** was responsible for the overall firmware development and system integration, including configuration and implementation of ADC, SPI, PWM,



Figure 9: Group Photo after Project Execution

GPIO, timer, and interrupt mechanisms. He developed the real-time data acquisition pipeline, implemented sensor calibration and filtering, performed system debugging in CCS, and authored the reports and schematics.

- **Sourish Kundu** contributed to the implementation of the I2C-based external LCD

interface and assisted in hardware interfacing, implementation of PWM, timer, and interrupt mechanisms, peripheral mapping, and system-level testing and validation. He also designed the novel pixelation-based visualization on the SPI TFT display and performed system debugging in CCS.

References

- 1 Class notes of Prof. S. Sastry, *Introduction to Embedded System Design (IN 278)*, IISc, Jan 2026.
- 2 Texas Instruments, *BOOSTXL-EDUMKII Educational BoosterPack User Guide*, SLAU599B.
- 3 Texas Instruments, *TMS320F28379D LaunchPad User Guide*, SPRUI77C.
- 4 Texas Instruments, *TMS320F2837xD Technical Reference Manual*, SPRUHM8K.
- 5 Datasheet of H1635 Soil Moisture Sensor.
- 6 Datasheet of 2.4-inch SPI TFT Display Module (ILI9341 / ILI9341-based).
- 7 NXP Semiconductors, *PCF8574 Remote 8-bit I/O Expander for I2C-bus Datasheet*.
- 8 Documentation for 16×2 LCD Module (HD44780 compatible).